

ArmorIQ Policy Isolation

Research & Strategy Report

Moving ArmorIQ's policy enforcement to a trust boundary that no AI agent can breach – even with sudo.

Covers: problem analysis, the sudo problem, hardware enclave evaluation, Kernel Assurance Plane (KAP) analysis, competitive landscape, architecture (with and without KAP), security deep dive, CBSE attack class, implementation strategy, and risk assessment.

Prepared by: ArmorIQ Engineering **Date:** June 2026 **Status:** Internal Review / Architecture Decision **Research basis:** Deep research across 24 sources, 119 claims extracted, 25 adversarially verified (18 confirmed, 7 refuted)

Table of Contents

- Part 1: Why We Need This
 - 1.1 The Fundamental Security Problem
 - 1.2 Current State
 - 1.3 Secure vs Security Theater
 - 1.4 Beyond ArmorClaude
 - 1.5 Why We Need the Daemon
- Part 2: The Sudo Problem
 - 2.1 The Problem
 - 2.2 What Sudo Actually Does
 - 2.3 Mitigations That Work Against Root
 - 2.4 The Defense Stack
- Part 3: Why Not Hardware Enclaves
 - 3.1 What Hardware Enclaves Are
 - 3.2 What They Protect Against
 - 3.3 Known Attacks and Broken Guarantees

- 3.4 Why Enclaves Don't Fit ArmorIQ
- 3.5 Where Enclaves Do Make Sense
- Part 4: PAP, IAP, KAP – Our Research Stack
 - 4.1 PAP: Intent-Preserving Refinement
 - 4.2 IAP: Execution-Level Cryptographic Verification
 - 4.3 KAP: Kernel Assurance Plane
 - 4.4 How KAP Maps to ArmorIQ
 - 4.5 Why We Wait on KAP Deployment
 - 4.6 Why We Design for KAP Now
 - 4.7 Architecture: With KAP vs Without KAP
- Part 5: What the Industry Does
 - 5.1 Competitive Architecture Comparison
 - 5.2 Industry Patterns That Work
 - 5.3 The CBSE Attack Class
 - 5.4 The Key Insight From Industry
- Part 6: Architecture Without KAP
 - 6.1 The ArmorIQ Policy Daemon
 - 6.2 How Policy Mutation Works
 - 6.3 How Enforcement Works
 - 6.4 ArmorClaude After Migration
 - 6.5 Cross-Product Architecture
 - 6.6 Enterprise Hardening
- Part 7: Architecture With KAP
 - 7.1 KAP Integration Architecture
 - 7.2 PAP-KAP Adapter for ArmorIQ
 - 7.3 IAP-KAP Adapter for CSRG
 - 7.4 What Changes, What Stays
- Part 8: Security Deep Dive
 - 8.1 Attack Vector Analysis
 - 8.2 Socket Authentication Model
 - 8.3 CBSE Protection
 - 8.4 Failure Modes
 - 8.5 ArmorIQ vs Industry
- Part 9: Implementation Strategy
- Part 10: Open Questions
- Part 11: Risk Assessment

Part 1: Why We Need This

1.1 The Fundamental Security Problem

ArmorClaude enforces policies on AI tool usage inside Claude Code. Today, the enforcement layer and the AI agent run as the same OS user on the same machine. This means:

- Every file ArmorClaude can read/write, Claude can too
- Every process ArmorClaude can spawn, Claude can too
- Every socket ArmorClaude can connect to, Claude can too
- Every credential ArmorClaude can access, Claude can too

Analogy: It's like hiring a security guard who shares the same keycard as the people they're guarding. The guard can check badges, but anyone being checked can also grab the guard's clipboard and rewrite the access list.

1.2 Current State

Over 7 phases, we built a comprehensive policy system with 195 automated tests:

Layer	What it does	Security strength
UserPromptSubmit hook + blockPrompt()	Human-only policy commands (/armor-policy)	Strong – architectural guarantee from Claude Code. LLM cannot forge this event.
Path guard (PreToolUse)	Blocks Write/Edit/Bash writes to policy files	Medium – catches obvious attempts, bypassable with obfuscation
Handler name guard	Blocks Bash commands referencing handleArmorPolicyCommand etc.	Medium – catches direct invocation, bypassable with encoding
Crypto digest (CSRG)	Detects policy.json tampering after the fact	Strong – cryptographic, but reactive not preventive
MCP deny-by-default	Unknown MCP servers blocked until approved	Strong – gate in PreToolUse, human approval via hook
OPA enforcement mode	Cloud policy evaluation with circuit breaker	Strong – but only when cloud is reachable

1.3 Secure vs Security Theater

Actually secure (architectural/cryptographic guarantees):

- UserPromptSubmit hook – Claude Code guarantees this fires only on human input
- blockPrompt() – consumes input before LLM sees it
- CSRG crypto digest – detects tampering mathematically
- policy_update MCP tool deletion – tool literally doesn't exist

Security theater (best-effort, bypassable by determined LLM):

- Path guard regex – Claude can obfuscate (base64 -d | node, string concatenation)
- Handler name blocking – Claude can import via dynamic import() with constructed string
- File permission checks – same OS user, Claude can chmod
- Any in-process guard – Claude has the same capabilities as the guard

Every guard we can build in-process, the LLM can theoretically unbuild. It's the same trust domain.

1.4 Beyond ArmorClaude

ArmorIQ has multiple AI tool products:

- **ArmorClaude** – Claude Code enforcement
- **ArmorCopilot** – GitHub Copilot enforcement (planned)
- **ArmorCodex** – OpenAI Codex enforcement (planned)
- **Custom agents** – via ArmorIQ SDK

A shared, isolated enforcement layer eliminates duplicate enforcement code, provides consistent policy across all AI tools, centralizes credential management, and enables org-wide policy from a single control plane.

1.5 Why We Need the Daemon

The enforcement daemon (`armoriq-policyd`) is the core of our architecture. It exists to solve five problems that no other approach can:

CBSE prevention. Configuration-Based Sandbox Escape (CVE-2026-25725, CVSS 7.7) is the #1 real-world attack on AI coding tools today. Agents modify their own config files (`.mcp.json` , `CLAUDE.md`) to inject malicious MCP servers or weaken restrictions. If

enforcement lives in the agent's process or config, the agent can change it. A separate daemon is immune – the agent's config changes don't affect enforcement.

Cross-product consistency. A developer using Claude Code + Copilot + Codex needs one policy, not three. The daemon is the only local process that can enforce policy across all three tools simultaneously.

Persistence. In-process enforcement dies when the agent exits. The daemon persists, maintains audit logs, syncs with the cloud backend, and is ready when any agent starts.

Local speed. CLI tool calls happen in milliseconds. A 50-200ms cloud round-trip per policy decision makes the tool unusable. The daemon provides <5ms local enforcement with async cloud sync.

Trust domain separation. Any enforcement running in the same process as the agent is in the same trust domain. The daemon creates a different trust domain – a separate process that the agent doesn't control.

The OS user separation (`_armoriq`) and MAC profiles are hardening on top of the daemon. They are not the reason the daemon exists.

Part 2: The Sudo Problem

2.1 The Problem

If the agent has sudo access, can it bypass policy files owned by a different OS user?

Yes. In many real-world environments, developers have passwordless sudo configured, CI/CD agents run as root, Docker containers run as root by default, and AI coding agents are given sudo for system tasks (installing packages, running services).

2.2 What Sudo Actually Does

sudo gives root. Root (UID 0) bypasses Unix Discretionary Access Control (DAC) entirely.

If an AI agent process can execute `sudo` , it can:

```

sudo cat /var/lib/armoriq/policy.json           # Read any file
sudo vi /var/lib/armoriq/policy.json           # Write any file
sudo kill $(pgrep armoriq-policyd)             # Kill the daemon
sudo chown aniket /var/lib/armoriq/policy.json # Take ownership
sudo chmod 777 /var/lib/armoriq/policy.json    # Open permissions
sudo su _armoriq                               # Become the daemon user
sudo rm -rf /var/lib/armoriq/                  # Delete everything

```

The separate OS user model (`_armoriq`) stops a non-root agent completely. But root is god mode on a standard Linux/macOS system – DAC provides zero protection against root.

The UK AI Safety Institute's SandboxEscapeBench (March 2026) found that frontier LLMs (Claude Opus 4.5, GPT-5) achieve ~50% success rates escaping misconfigured containers. If given sudo, policy evasion would be trivial.

2.3 Mitigations That Work Against Root

Strip sudo from the agent:

```

# Via systemd (for daemon-launched agents):
[Service]
CapabilityBoundingSet=
NoNewPrivileges=true
Environment=PATH=/usr/bin:/bin

```

`NoNewPrivileges=true` prevents any child process from gaining new privileges via `setuid/setgid` binaries – even if the agent finds `/usr/bin/sudo`, the kernel refuses the escalation.

Mandatory Access Control (stops root):

MAC systems override DAC. Even UID 0 is subject to MAC policy.

Mechanism	Stops root?	Platform
SELinux type enforcement	YES	RHEL, Fedora, CentOS
AppArmor profile	YES	Ubuntu, Debian, SUSE
macOS sandbox profile	YES	macOS

SELinux has demonstrably stopped real privilege escalation CVEs: - **CVE-2019-9213** (mmap NULL pointer deref): SELinux enforcing mode caused segfault instead of escalation - **CVE-2019-13272** (ptrace escalation): Blocked when `deny_ptrace` boolean enabled

Source: Red Hat Article 6964380

Root CAN disable MAC systems (`setenforce 0` for SELinux). This is prevented by Linux kernel lockdown (since 5.4), which blocks root from accessing sensitive kernel interfaces. Secure Boot + kernel lockdown together prevent loading unsigned modules.

Source: man7.org `kernel_lockdown(7)`

2.4 The Defense Stack

The defense has two tiers: a base that ships to everyone, and an enterprise hardening add-on.

Base tier (everyone – individual devs and enterprise):

```
Separate process daemon (armorq-policyd)
- Runs as _armorq system user
- Policy files owned by _armorq (mode 0600)
- Agent communicates via Unix socket (read-only enforcement queries)
- Policy mutations only via human-triggered auth: 'user-session'
- CSRG Merkle proofs detect any tampering
- Cloud sync provides remote source of truth
```

Stops: all non-root policy tampering, CBSE attacks, cross-agent escalation, config file injection

Enterprise hardening tier (for compliance-driven teams):

```
MAC profile (ONE of: SELinux on RHEL, AppArmor on Ubuntu, Sandbox on macOS)
- Single config file dropped by installer
- Stops even root/sudo from touching policy files
```

Additionally stops: root-level policy tampering

Two tiers. That's it.

For individual developers, the base tier is sufficient. Devs don't typically give AI agents sudo access, and the daemon separation prevents all realistic attack vectors.

For enterprise teams with compliance requirements, the enterprise tier adds the MAC profile. Security auditors want to see "even root can't tamper with policy" – one config file gives them that answer.

Part 3: Why Not Hardware Enclaves

3.1 What Hardware Enclaves Are

Hardware enclaves create a Trusted Execution Environment (TEE) where code and data are protected by the CPU itself. Even root, the kernel, and the hypervisor cannot read the enclave's memory.

Technology	Vendor	What it protects against	Current status
Intel SGX	Intel	Root, kernel, hypervisor, physical access	Deprecated on consumer CPUs (12th gen+); server Xeons only
Intel TDX	Intel	Hypervisor, other VMs (VM-level, not process-level)	Available on 4th gen Xeon Scalable+
AMD SEV/SEV-SNP	AMD	Hypervisor, other VMs, physical access	Available on EPYC 3rd gen+
ARM TrustZone	ARM	Normal-world OS (two-world isolation)	Mobile/embedded, not developer workstations
Apple Secure Enclave	Apple	Main CPU, kernel (stores keys, biometrics)	macOS/iOS, very limited API (keychain only)

3.2 What They Protect Against

An SGX enclave could theoretically seal the policy JSON with an enclave-specific key derived from CPU fuses, only unseal and evaluate inside the enclave, and ensure the unsealed policy never exists in main memory accessible to the OS. Even root, even the kernel, even a hypervisor cannot read the enclave's memory pages.

This is the strongest possible isolation – data is protected by hardware, not software.

3.3 Known Attacks and Broken Guarantees

The security guarantee of enclaves has been significantly eroded by real-world attacks:

Attack	Year	CVE	What broke	Impact
Foreshadow / L1TF	2018	CVE-2018-3615	L1 cache side-channel	Could read enclave memory
Plundervolt	2019	CVE-2019-11157	Voltage glitching corrupts SGX	Could forge attestation
SGAxe	2020	-	Evolved Foreshadow; broke attestation key	Could impersonate any enclave
CacheOut	2020	CVE-2020-0549	L1 data eviction side-channel	Reads enclave data
AEPIC Leak	2022	CVE-2022-21233	Architectural bug leaking via APIC	Reads SGX data without side-channel
Downfall	2023	CVE-2022-40982	Gather Data Sampling	Reads across SGX boundary
LeftoverLocals	2024	-	GPU residual-state leakage	Affects GPU TEEs
DDR5 bus attack	2025	-	Sub-\$1000 physical memory bus tap	Breaks SGX, TDX, SEV-SNP
Spectre variants	Ongoing	Multiple	Speculative execution leaks	Broad microarchitectural issue

From a May 2026 Imperial College London survey paper (arxiv:2605.03213v1):

"Side-channel attacks including timing leakage, cache and bus contention, page-fault or controlled-channel attacks, speculative-execution effects, GPU residual-state leakage...may still expose information to a sufficiently capable adversary."

3.4 Why Enclaves Don't Fit ArmorIQ

SGX is deprecated on consumer CPUs. Developers won't have it on their laptops. SGX survives only on Xeon server processors.

macOS has no general-purpose enclave. Apple Secure Enclave stores keys and biometrics. We cannot run policy evaluation code inside it. We are cross-platform.

Development complexity is extreme. Enclave code must be written in restricted C/C++. No system calls, no dynamic memory. Policy evaluation that takes 50 lines of JavaScript would require 500+ lines of enclave-compatible C.

Side-channel attacks keep being found. The security guarantee degrades over time rather than improving.

Enclaves cannot prevent semantic attacks. From the Imperial College survey: “TEE-backed memory isolation can keep prompts, retrieved context, and runtime state confidential from a privileged infrastructure adversary, but it does not by itself stop a prompt-injected agent from leaking that data through its own outputs or tool calls.”

No production framework exists. “No broadly established end-to-end framework yet binds them into a coherent security substrate for production agentic AI.”

Overkill for the threat model. We are protecting policy JSON from an AI coding agent, not nuclear secrets from nation-state attackers.

3.5 Where Enclaves Do Make Sense

Enclaves are valuable for specific hardening, not primary enforcement:

Use Case	Technology	Timeline
CSRG signing key storage	Apple Secure Enclave / TPM 2.0	Future
Cloud backend isolation	AMD SEV-SNP / Intel TDX	Future
Credential storage	OS keychain (hardware-backed)	Near-term

We use MAC-based enforcement (SELinux/AppArmor) for root-resistant policy protection on Linux today, sandbox profiles on macOS, and hardware security only where it fits naturally – key storage. When KAP matures, we add kernel-level authority-lineage enforcement without hardware dependency.

Part 4: PAP, IAP, KAP – Our Research Stack

We have three research artifacts that form a coherent stack. Each operates at a different layer.

4.1 PAP: Intent-Preserving Refinement

PAP is a theoretical model, not a daemon or service. The PAP paper defines:

- A **refinement chain**: $H \rightarrow I \rightarrow P \rightarrow A \rightarrow E$ (Human purpose $\rightarrow$ Intent $\rightarrow$ Plan $\rightarrow$ Actions $\rightarrow$ Effects)
- A **refinement invariant**: at each step, representation uncertainty must decrease and authority must only narrow (never widen)
- An **authority lattice**: $A = (T, S, E)$ where T = available tools, S = active constraints, E = execution context
- A **cone of intent**: uncertainty narrows from purpose to execution

PAP governs how plans should evolve. The key insight: correctness is not about whether an individual action was authorized, but whether the plan that produced it was a faithful refinement of the user's original purpose.

4.2 IAP: Execution-Level Cryptographic Verification

IAP provides execution-level guarantees – ensuring that actions follow the committed plan using cryptographic structures to bind execution to reasoning. Our CSRG (Cryptographic State Reference Graph) Merkle tokens are the implementation of IAP concepts. They detect tampering mathematically and bind policy state to a verifiable chain.

4.3 KAP: Kernel Assurance Plane

KAP is a Linux kernel enforcement layer. It addresses a gap that neither PAP nor IAP can close:

"Linux can decide whether an individual syscall, credential, or object access is permitted, but it does not decide whether the authority that produced that operation is still consistent with the task that authorized the agent."

Example: An agent told to "refactor a web server config" synthesizes `fork()` $\rightarrow$ `execve("curl")` $\rightarrow$ `connect(external_host)` $\rightarrow$ `send(config_archive)`. Each syscall is individually legal. But the combined execution exfiltrates configuration data. The authority drifted beyond the task.

KAP's solution: semantic authority refinement happens in user space (where prompts and context are available); deterministic lineage enforcement happens in the kernel.

Core KAP concepts:

Concept	What it is
Intent Lineage Group (igroup)	Kernel-resident isolation unit. Tracks authority boundary, lineage root, epoch, task membership. Survives fork/execve/setns.
Authority Epoch	Versioned, immutable authority snapshot. Stale epochs fail closed instantly. Authority can only narrow, never widen.
igfd	igroup file descriptor. Carries igroup + epoch + lineage root. Epoch revocation kills stale handles.
Authority Snapshot	Immutable predicate set for one epoch. Typed predicates over exec, file, net, namespace, cred, IPC, descriptor, procfs, execmem, BPF, ptrace, io_uring.
kapfs	Pseudofilesystem at <code>/sys/fs/kap/<igroup>/</code> . Only writable by privileged kapd daemon.
kapd	User-space daemon that validates artifacts and writes compiled state to kapfs.
Transition Capability	Authorized lineage-changing event. Prevents replay via sequence numbers.

KAP is a kernel overlay on Linux 6.8.12 – three new files (~1,771 lines of kernel code) wired into the LSM and VFS frameworks. This is the same approach used to add SELinux, AppArmor, and Landlock to Linux. An RFC patch series is ready for upstream Linux submission.

Component	Lines of code
Kernel enforcement (LSM + kapfs)	1,771
User-space runtime (kapd, kapctl)	1,275
PAP-KAP and IAP-KAP adapters	571

4.4 How KAP Maps to ArmorIQ

Our daemon provides the data that feeds KAP's kernel enforcement:

ArmorIQ Concept	KAP Equivalent
<code>armoriq-policyd</code> daemon	Feeds PAP-KAP adapter + kapd. Daemon outputs policy as typed predicates; adapter compiles to kernel format.
CSRG Merkle tokens	IAP lineage root + Merkle commitment. Our Merkle chain maps directly.
Policy version/epoch	Authority epoch
Tool allowlist (Read, Bash, etc.)	Exec + file + IPC predicates
<code>/armor policy confirm</code>	IAP Intent Commit. Human confirmation = lineage commitment event.
<code>/armor policy add</code>	IAP Trust Update. Policy mutation = epoch-advancing transition.
Product scope (armorclaude/copilot)	igroup per product

The KAP paper states: "The PAP-KAP adapter can be realized today by compiling existing runtime metadata into KAP predicates. Agent frameworks already maintain tool registries, MCP schemas, command allowlists, skill constraints, workspace roots, credential bindings, deployment targets, and network policy."

This describes our daemon's output exactly.

4.5 Why We Wait on KAP Deployment

1. **Customer deployment burden:** KAP requires a kernel overlay. Customers can't `brew install` it.
2. **Linux only:** KAP doesn't work on macOS. We are cross-platform.
3. **Maturity:** The prototype runs in a QEMU VM. Not yet battle-tested.
4. **Upstream timeline:** KAP is positioned for mainline Linux inclusion. Waiting is pragmatic.
5. **The existing toolbox works:** A single MAC profile (SELinux or AppArmor) solves the sudo problem on Linux today.

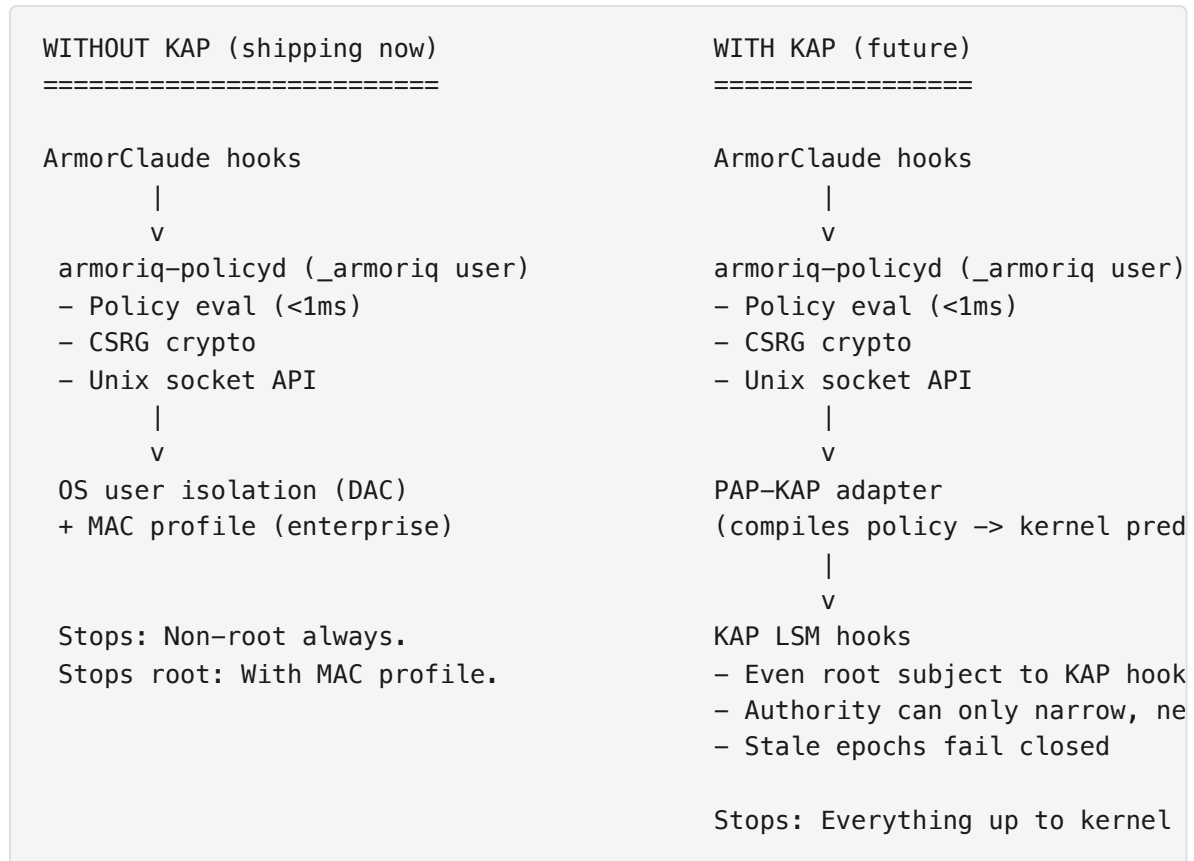
4.6 Why We Design for KAP Now

We design `armoriq-policyd` with KAP-compatible interfaces so that integration later is an adapter, not a rewrite:

- Policy internally represented as typed predicates (exec, file, net, IPC, cred) even though current enforcement returns allow/deny
- CSRG tokens are epoch-aware – our Merkle chain already maps to IAP lineage roots

- Socket protocol includes product ID (maps to igroup), policy version (maps to epoch), and tool context (maps to predicate resolution)

4.7 Architecture: With KAP vs Without KAP



The without-KAP path is complete and shippable. KAP is an additive enhancement, not a replacement. Nothing is thrown away.

Part 5: What the Industry Does

5.1 Competitive Architecture Comparison

Product	Enforcement Location	How Agent Can't Modify Policy	Latency	License
Invariant Guardrails	Gateway proxy	Agent's URLs rewrite to proxy	Near-zero	Open source
Lasso Security	MCP Gateway proxy	External gateway intercepts MCP	Real-time	Commercial

Product	Enforcement Location	How Agent Can't Modify Policy	Latency	License
		tool calls		
Lakera Guard	Cloud API / self-hosted	API-based; agent sends payloads to external service	Sub-50ms	Commercial (Cisco)
Arthur AI Shield	Federated control/data plane	Control plane in cloud; data plane in VPC	p95 <200ms	Commercial
OPA / Styra	Sidecar PDP	Sidecar has own filesystem; policies from bundle server	Sub-millisecond	Open source
Permit.io / OPAL	Sidecar PDP (OPA + OPAL)	OPAL server pushes diffs via pub/sub	1-5ms	OPAL OSS
Cerbos	Sidecar / service	Stateless PDP loads from git/S3	Sub-millisecond	Open source
Guardian Shell	Landlock + seccomp + eBPF + cgroups	Kernel-level enforcement per agent	Low	Commercial
Tetragon	eBPF in-kernel enforcement	eBPF programs enforce policy at syscall level	~0.7% CPU	Open source
Anthropic built-in	In-model (training-time)	Weights immutable at inference	Zero	Built into Claude
ArmorIQ	System user daemon + MAC	OS isolation + MAC + CSRG crypto	<5ms	Proprietary

Guardian Shell's claims about "solving" TOCTOU races and symlink attacks were refuted in our adversarial verification (0-3 votes). Tetragon has genuine eBPF enforcement at ~0.7% overhead but its claim of "eliminating TOCTOU vectors" was also refuted (0-3). Known bypasses exist via `writenv()`, `io_uring`, and bash builtins.

5.2 Industry Patterns That Work

Gateway/Proxy (Invariant, Lasso): Physical interposition. Agent can't skip what it can't see around.

Sidecar PDP (OPA, Cerbos, Permit.io): Separate process, separate filesystem, policies from external authority. Policy authoring and evaluation are in separate trust domains. This is the pattern our daemon follows.

Cloud API (Lakera, Arthur): No local state to tamper with, but 50-200ms latency and requires internet.

5.3 The CBSE Attack Class

Configuration-Based Sandbox Escape (CBSE), coined by Cymulate Research Labs, is the most important emerging vulnerability class for our product:

"The sandbox isolation can be completely bypassed by modifying trusted files or execution paths that are later processed outside the sandbox."

Agents modify their own config files to escape isolation:

- `.mcp.json` – inject malicious MCP servers
- `CLAUDE.md` – inject malicious instructions
- `.vscode/settings.json` – modify tool behavior
- `.claude/settings.json` – modify permissions

Real CVEs:

CVE	Product	CVSS	Attack
CVE-2026-25725	Claude Code	7.7 (NVD: 10.0)	MCP config injection
CVE-2025-59536	AI agent tool	8.7	Config manipulation sandbox escape
CVE-2026-21852	AI agent tool	-	Config-based privilege escalation
CVE-2026-30615	AI agent tool	-	Configuration injection

Cross-agent escalation has been demonstrated: GitHub Copilot, hijacked via prompt injection, modifying Claude Code's `.mcp.json` to add a malicious MCP server enabling arbitrary code execution.

"This isn't theoretical. With current tools and defaults, it's very possible today and not well mitigated across the board." – Johann Rehberger (wunderwuzzi)

ArmorIQ prevents CBSE by design. Policy lives in the daemon, not in agent config files. The agent's config changes don't affect enforcement. MCP registry lives in the daemon. This is our strongest market differentiator.

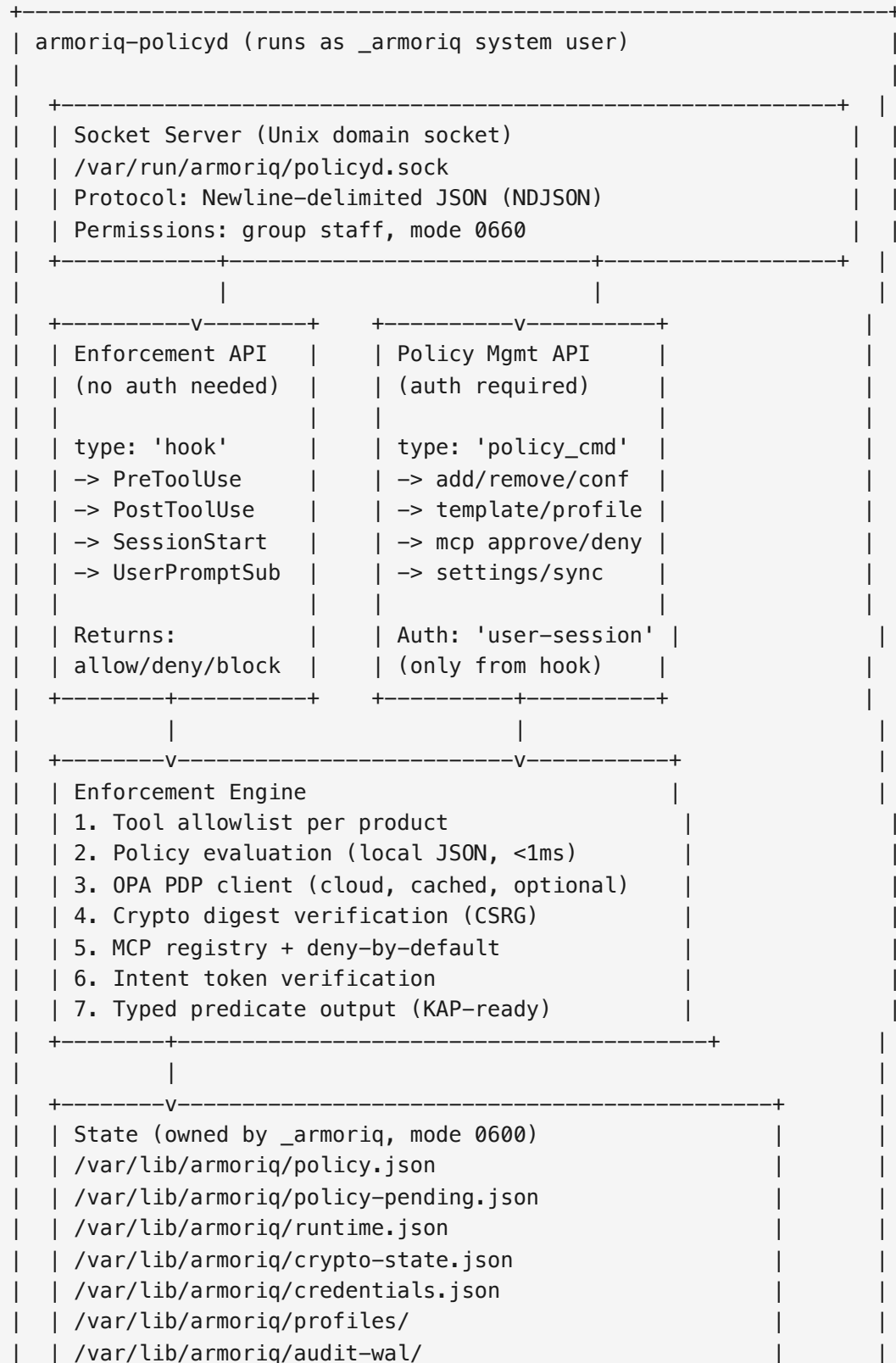
5.4 The Key Insight From Industry

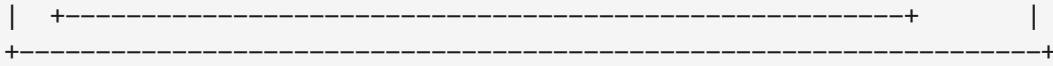
Every production-grade enforcement system separates policy decisions from policy administration. The agent can query for decisions but never change policy.

ArmorIQ is unique in providing: 1. Purpose-built enforcement for AI tools (not generic authz) 2. Human-only policy mutation via Claude Code's UserPromptSubmit guarantee 3. Cryptographic tamper detection via CSRG Merkle proofs 4. CBSE prevention by architectural separation 5. Multi-AI-tool support from a single daemon 6. Offline-first with optional cloud sync 7. KAP-compatible design for future kernel enforcement

Part 6: Architecture Without KAP

6.1 The ArmorIQ Policy Daemon





The enforcement engine outputs typed predicates (exec, file, net, IPC, cred) internally even though the current enforcement path returns allow/deny. This makes a future KAP adapter a thin translation layer.

6.2 How Policy Mutation Works

```

Human types /armor-policy add deny Bash
      |
      v
UserPromptSubmit hook fires (Claude Code guarantee: human-only)
      |
      v
Hook process (runs as user):
  1. Connects to /var/run/armoriq/policyd.sock
  2. Sends: {type:'policy_cmd', command:'/armor-policy add deny Bash',
            auth:'user-session', reqId:'r-1'}
  3. Daemon validates auth tag (only UserPromptSubmit sets this)
  4. Daemon stages change to policy-pending.json (it owns the file)
  5. Returns proposal text
  6. Hook calls blockPrompt(output) -> Claude LLM never sees this

```

Why Claude can never do this: 1. Claude can't type in the terminal – can't trigger UserPromptSubmit 2. Claude can't set `auth: 'user-session'` – daemon rejects without it 3. Even if Claude connects to socket directly, it can only send enforcement queries 4. Claude can't write to `/var/lib/armoriq/policy.json` – wrong user, permission denied 5. Claude can't kill the daemon – wrong user; launchd/systemd restarts it 6. With sudo + MAC profile: still can't – MAC overrides DAC

6.3 How Enforcement Works

```

Claude wants to call Bash('rm -rf /')
  |
  v
Claude Code fires PreToolUse hook
  |
  v
Hook process:
  1. Connects to /var/run/armoriq/policyd.sock
  2. Sends: {type:'hook', event:'PreToolUse', product:'armorclaud',
            input:{tool_name:'Bash', tool_input:{command:'rm -rf /'}}}
  3. Daemon evaluates policy (local JSON, <1ms)
  4. Returns: {permissionDecision:'deny',
              reason:'ArmorClaude: Bash is denied by policy'}
  5. Hook relays to Claude Code -> tool call blocked

Latency: <5ms total (socket connect + JSON + eval + response)

```

6.4 ArmorClaude After Migration

ArmorClaude shrinks from ~2000 lines of enforcement logic to ~200 lines of client code:

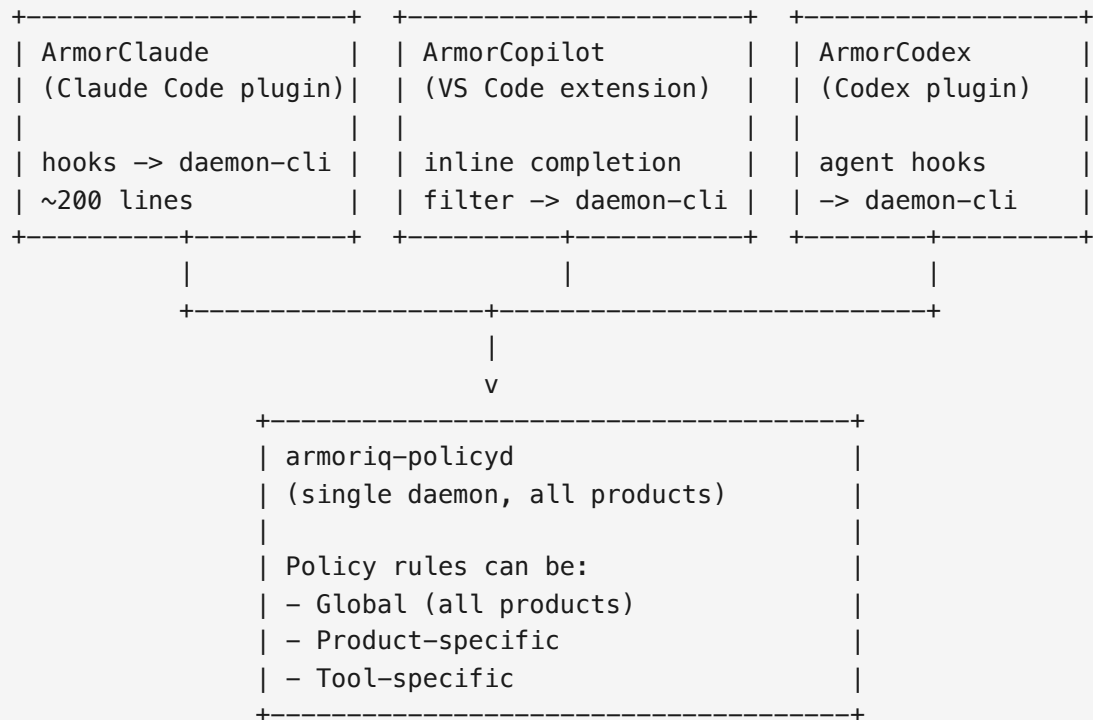
```

armorclaud/
+-- hooks/hooks.json
+-- skills/armor-policy/SKILL.md
+-- scripts/
|   +-- bootstrap.mjs
|   +-- hook-router.mjs      # THIN CLIENT: socket -> daemon
|   +-- policy-mcp.mjs      # MCP server (reads via daemon)
|   +-- lib/
|       +-- daemon-client.mjs # Socket connect + NDJSON
+-- package.json

```

Everything else moves to armoriq-policyd.

6.5 Cross-Product Architecture



6.6 Enterprise Hardening

For enterprise customers who need sudo protection, the installer drops a single MAC config file:

SELinux (RHEL/Fedora):

```

# armoriq.te
policy_module(armoriq, 1.0)

type armoriq_daemon_t;
type armoriq_policy_t;
type ai_agent_t;

init_daemon_domain(armoriq_daemon_t, armoriq_daemon_exec_t)

allow armoriq_daemon_t armoriq_policy_t:file { read write create rename unl
allow ai_agent_t armoriq_var_run_t:unix_stream_socket connectto;

/var/lib/armoriq(/.*)?    gen_context(system_u:object_r:armoriq_policy_t,s0
/var/run/armoriq(/.*)?    gen_context(system_u:object_r:armoriq_var_run_t,s

```

AppArmor (Ubuntu/Debian):

```
profile ai-agent /usr/bin/claude flags=(enforce) {
    owner /home/** rw,
    /tmp/** rw,
    /usr/bin/* ix,

    deny /var/lib/armoriq/** rwk!x,
    deny /usr/local/lib/armoriq/** w!k!x,

    /var/run/armoriq/policyd.sock rw,
}
```

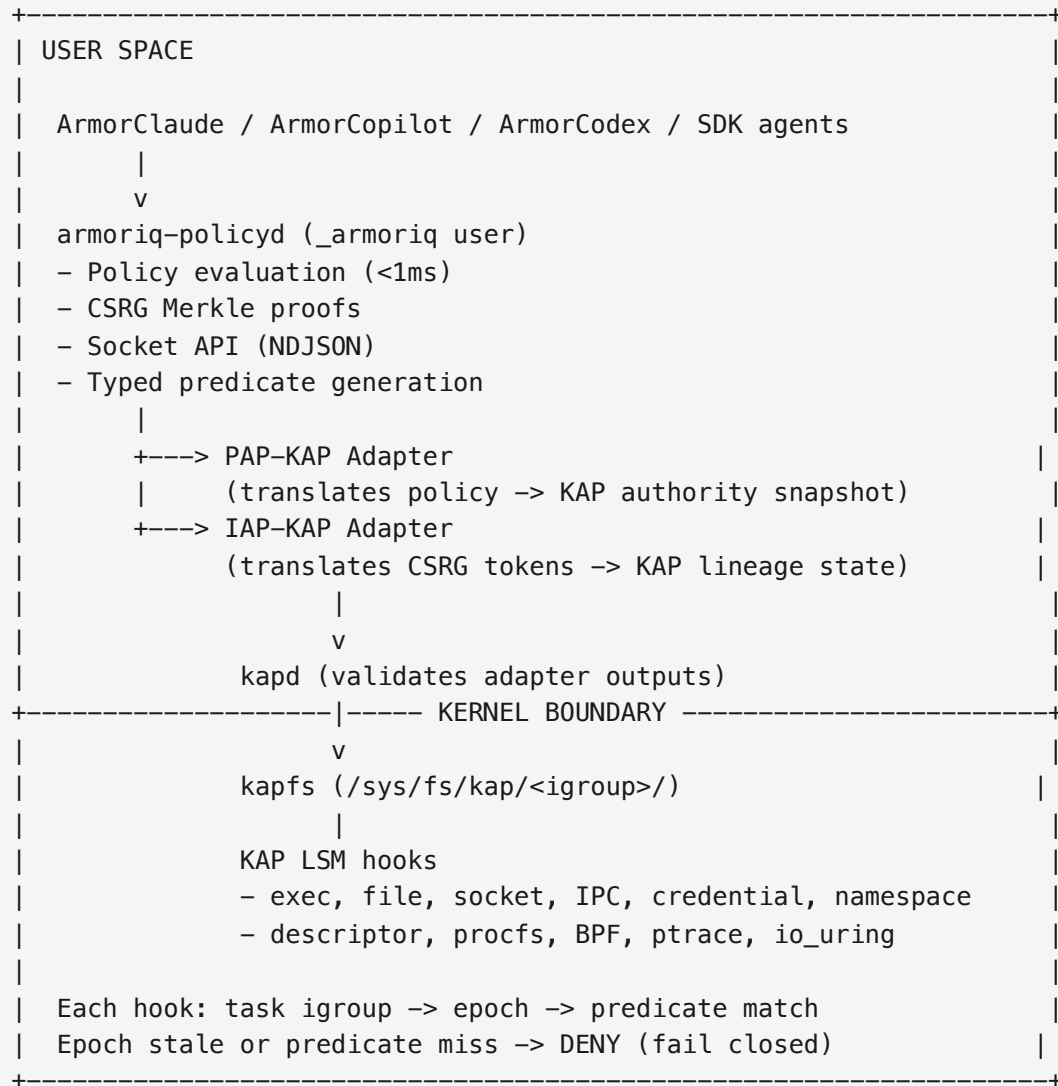
macOS launchd:

```
<plist version="1.0">
<dict>
    <key>Label</key>
    <string>io.armoriq.policyd</string>
    <key>ProgramArguments</key>
    <array>
        <string>/usr/local/lib/armoriq/policyd</string>
    </array>
    <key>UserName</key>
    <string>_armoriq</string>
    <key>RunAtLoad</key>
    <true/>
    <key>KeepAlive</key>
    <true/>
</dict>
</plist>
```

Part 7: Architecture With KAP

7.1 KAP Integration Architecture

When KAP matures, the integration adds a kernel enforcement layer below the existing daemon:



7.2 PAP-KAP Adapter for ArmorIQ

ArmorIQ Policy Statement =====		KAP Predicate(s) =====
permit tool Read	-->	file: allow read on workspace
permit tool Grep	-->	exec: allow /usr/bin/grep
forbid tool Write	-->	file: deny write on workspace
forbid tool Bash	-->	exec: deny /bin/bash, /bin/sh
permit Bash when program in [ls]	-->	exec: allow /bin/ls
deny MCP server evil-server	-->	ipc: deny unix socket to evil-ser
workspace scope: /home/dev/project	-->	file: allow r/w under /home/dev/p

7.3 IAP-KAP Adapter for CSRG

ArmorIQ CSRG Event =====		KAP Lineage State =====
Policy created (initial)	-->	lineage root = Merkle(policy_v1) epoch = 1
/armor policy confirm <proposal>	-->	new epoch = epoch + 1 new root = Merkle(policy_v2)
/armor policy add deny Bash	-->	authority narrows (Bash removed)
Policy tamper detected	-->	epoch revocation -> deny-all

7.4 What Changes, What Stays

Component	Without KAP	With KAP	Changes?
ArmorClaude hooks	Thin client -> daemon	Same	No
armoriq-policyd	Policy eval + enforcement	Policy eval + adapter output	Minor
Socket protocol	NDJSON allow/deny	Same + predicate metadata	Additive
Policy JSON format	armor.policy.v1	Same	No
CSRG Merkle tokens	Tamper detection	+ lineage roots	Reuse
Enforcement location	Daemon (user space)	Daemon + kernel	Additive
macOS support	Daemon + sandbox	Same (no KAP on macOS)	No

Nothing is thrown away. KAP is additive.

Part 8: Security Deep Dive

8.1 Attack Vector Analysis

#	Attack	Current (same user)	With daemon + MAC	With KAP
1	Write to policy.json via Edit/Write	Path guard (bypassable)	BLOCKED	BLOCKED
2	Write to policy.json via Bash	Regex guard (bypassable)	BLOCKED	BLOCKED
3	Import handler via Bash	Name regex (bypassable)	BLOCKED	BLOCKED
4	Base64-encode modifying script	No guard	BLOCKED	BLOCKED
5	Kill enforcement daemon	Same user, works	BLOCKED	BLOCKED
6	Read API key from credentials	Same user, works	BLOCKED	BLOCKED
7	Modify daemon code/config	Same user, works	BLOCKED	BLOCKED
8	Send fake mutation via socket	Can send anything	BLOCKED	BLOCKED
9	chmod policy files	Same user, works	BLOCKED	BLOCKED
10	Sudo to escalate	Requires password	BLOCKED (MAC)	BLOCKED (kernel)
11	Modify own config (CBSE)	No guard	BLOCKED	BLOCKED
12	Inject malicious MCP server	MCP registry in agent	BLOCKED	BLOCKED
13	Cross-agent escalation	Possible	BLOCKED	BLOCKED
14	Prompt injection in tool output	No effect	Same	Same

#	Attack	Current (same user)	With daemon + MAC	With KAP
15	Social-engineer user	Human decision	Same	Same
16	Authority drift (legal ops)	No detection	No detection	BLOCKED (KAP unique)

Current: 4/16 blocked. Daemon + MAC: **13/16**. Daemon + MAC + KAP: **14/16**.

8.2 Socket Authentication Model

The daemon socket is accessible to the staff group (mode 0660). Claude CAN connect. Here's why that's secure:

Read operations (no auth): enforcement queries, policy listing, health check. These are fine – getting policy decisions is the whole point.

Write operations (auth required): The `auth: 'user-session'` tag is set by the UserPromptSubmit hook handler, which only fires on human terminal input. The daemon additionally verifies peer credentials (SO_PEERCREDS on Linux / LOCAL_PEERCREDS on macOS) and validates that the connecting PID is a child of Claude Code's hook spawn.

8.3 CBSE Protection

CBSE Attack Flow (vulnerable systems):

Agent -> modifies `.mcp.json` -> next session loads malicious MCP -> code e

ArmorIQ Defense:

Agent -> modifies config -> irrelevant

Enforcement happens in daemon, not in agent config

Daemon reads policy from `/var/lib/armoriq/` (owned by `_armoriq`)

MCP registry lives in daemon, not in agent config files

8.4 Failure Modes

Scenario	Behavior	Why correct
Daemon not installed	Observe-only mode	User chose not to install
Daemon crashes	launchd/systemd restarts; fail-closed during gap	Better to block briefly than allow everything

Scenario	Behavior	Why correct
Socket file deleted	Daemon recreates on restart; fail-closed	Transient, auto-recovers
Backend unreachable	Local enforcement continues; audit WAL queues	Offline-first
Policy file corrupted	Crypto digest mismatch -> deny-all + alert	Tamper detection
OPA unreachable	Circuit breaker -> local JSON	Graceful degradation

8.5 ArmorIQ vs Industry

Capability	OPA/Styra	Cerbos	Invariant	Guardian Shell	ArmorIQ
Stops root/sudo	No	No	No	Partial	YES (MAC)
CBSE protection	No	No	No	Partial	YES
Authority lineage	No	No	No	No	Future (KAP)
Latency	Sub-ms	Sub-ms	Near-zero	Low	<5ms
Offline	Yes	Yes	No	Yes	Yes
Human-only mutation	N/A	N/A	N/A	Partial	Yes
Crypto tamper detection	Bundle signing	No	No	No	CSRG Merkle
Cross-platform	Yes	Yes	Yes	Linux only	macOS + Linux

Part 9: Implementation Strategy

Phase	What	Delivers
Phase 0 (now)	Ship current architecture as v0.3	Working policy, 195 tests, immediate value

Phase	What	Delivers
Phase 1	Create armoriq-policyd repo, extract daemon	Standalone daemon with socket API
Phase 2	System service installer (launchd + systemd)	OS-level isolation, _armoriq user
Phase 3	ArmorClaude thin client migration	ArmorClaude becomes ~200 lines
Phase 4	Socket auth hardening + MAC profiles	Peer creds; SELinux/AppArmor/macOS sandbox
Phase 5	Cross-product client library	@armoriq/policy-client npm + Python
Phase 6 (future)	KAP integration	PAP-KAP + IAP-KAP adapters, kernel enforcement

armoriq-policyd (new repo):

```

armoriq-policyd/
+-- src/
|   +-- daemon.mjs, engine.mjs, policy.mjs
|   +-- armor-policy-commands.mjs
|   +-- policy-templates.mjs, policy-profiles.mjs
|   +-- policy-compiler.mjs, predicate-generator.mjs
|   +-- tool-registry.mjs, opa-client.mjs
|   +-- crypto-policy.mjs, backend-client.mjs
|   +-- runtime-state.mjs, audit-wal.mjs, fs-store.mjs
+-- install/
|   +-- install.sh
|   +-- io.armoriq.policyd.plist (macOS)
|   +-- armoriq-policyd.service (Linux)
|   +-- armoriq.te (SELinux), armoriq.apparmor, armoriq-agent.sb (macOS)
+-- tests/
+-- package.json

```

Installer experience:

```

curl -fsSL https://install.armoriq.ai/policyd | sudo bash
# Or: brew install armoriq-policyd

# Verify:
armoriq-policyd status
# -> ArmorIQ Policy Daemon v1.0.0 | running | uptime: 5s

```

Part 10: Open Questions

1. Should the daemon support Windows?
2. Homebrew formula to avoid curl-pipe-sudo friction?
3. Admin dashboard for policy management?
4. Org-wide policy distribution: backend push, daemon poll, or git-based?
5. Break glass: emergency override with MFA?
6. KAP upstream timeline tracking?

Part 11: Risk Assessment

Risk	Impact	Likelihood	Mitigation
sudo installation friction reduces adoption	High	Medium	Homebrew formula; observe-only mode without daemon
macOS Gatekeeper blocks unsigned daemon	Medium	High	Apple Developer ID signing; Homebrew distribution
Socket permission misconfiguration	High	Low	Installer validates; daemon refuses to start if insecure
MAC misconfiguration leaves gap	High	Medium	Installer auto-detects and configures
Version mismatch (client vs daemon)	Medium	Medium	Version handshake; client warns
CBSE attack on ArmorIQ itself	High	Low	Daemon owns all config; agent config irrelevant
Competitor ships similar product faster	Medium	Medium	Ship v0.3 immediately; daemon is v1.0
KAP never goes upstream	Medium	Medium	Without-KAP stack is complete and sufficient

Appendix A: Research Sources

Findings verified through adversarial verification (3-vote panels, 2/3 refute to kill):

1. Datadog Security Labs – Container Security Fundamentals Part 5
2. Red Hat Article 6964380 – SELinux as Security Pillar
3. man7.org – kernel_lockdown(7)
4. arxiv:2605.03213v1 – Imperial College London TEE Survey, May 2026
5. Tetragon project – eBPF enforcement capabilities
6. Johann Rehberger – Cross-Agent Privilege Escalation
7. Cymulate Research Labs – CBSE discovery
8. UK AI Safety Institute – SandboxEscapeBench, March 2026
9. arxiv:2601.11893v1 – LLM Agent Privilege Escalation Taxonomy
10. KAP Paper – Bhardwaj, Georgia Tech

Appendix B: Glossary

Term	Definition
DAC	Discretionary Access Control – Unix file permissions. Root bypasses.
MAC	Mandatory Access Control – SELinux/AppArmor. Root is subject to policy.
PAP	Intent-Preserving Refinement – our theoretical model for governing how plans evolve under uncertainty
IAP	Intent Assurance Plane – execution-level cryptographic verification
KAP	Kernel Assurance Plane – kernel-level authority lineage enforcement
igroup	Intent Lineage Group – KAP's unit of isolation
CSRG	Cryptographic State Reference Graph – our Merkle-based tamper detection
CBSE	Configuration-Based Sandbox Escape – attack via modifying agent config files
NDJSON	Newline-delimited JSON – wire protocol for daemon socket

Term	Definition
TEE	Trusted Execution Environment – hardware-isolated execution